DATABASE MANAGEMENT SYSTEMS

Complete Exam Notes for AKTU

12 Important Questions | Beginner-Friendly | 2-Mark and 10-Mark Answers

What is inside every question

- **Understand It First:** the idea in plain English with a real-life comparison. No jargon without a definition.
- **2-Mark Answer:** a short boxed answer you can write as is.
- **10-Mark Answer:** the full write-up with definitions, steps, tables, diagrams and examples.
- **Real-Life Use:** where this is used in real software (so it sticks).
- **Exam Tips:** what to write and what mistakes to avoid.

How to score full marks

- **2-mark question:** write only the green 2-Mark box. Four to five lines, key terms underlined.
- **10-mark question:** write the 2-mark answer first as your opening, then add the points, an example or table, and a neat diagram.
- Use headings, underline keywords, draw tables and diagrams with a ruler. Examiners give marks for each correct point they can find quickly.
- Practise each worked problem (Questions 10, 11, 12) by covering the answer and solving it again.

Four words you will see everywhere

- **Table (relation):** data arranged in rows and columns.
- **Row (tuple):** one record, for example one student.
- **Column (attribute):** one property, for example Name.
- **Key:** a column (or set of columns) that identifies a row uniquely, for example RollNo.

Contents

No.	Topic	Type
1	DDL vs DML	Theory + SQL
2	Trigger	Theory + SQL
3	Advantages of Normalization	Theory
4	Types of Functional Dependencies	Theory
5	Constraints	Theory + SQL
6	Relational Algebra and its Operations	Theory + examples
7	ACID Properties	Theory
8	Tuple, Relational Calculus, TRC and DRC	Theory + examples
9	1NF, 2NF, 3NF, BCNF	Theory + examples
10	Minimal Cover (numerical)	Solved problem
11	Serializability of a Schedule (numerical)	Solved problem
12	Lossless Decomposition (numerical)	Solved problem
	Quick Revision Sheet	Last-minute revision

Question 1: DDL vs DML

> **The question (typos fixed)**
>
> List the differences between DDL and DML.
>
> **Unit:** SQL and Database Languages

1. Understand It First (Plain English)

> **Think of building a house**
>
> A database is like a house. **DDL** is the work of the **architect and builders**: they build rooms, add a wall, or demolish a room. They change the **structure**.
>
> **DML** is the work of the **people living inside**: they put furniture in, move it, throw it out, or look at what is there. They work with the **contents**.
>
> In database words: DDL changes the **table design** (columns, types, names). DML changes the **data (rows)** stored inside the tables.

2. The 2-Mark Answer (write this)

> **2-Mark Answer**
>
> **DDL (Data Definition Language)** is used to create and change the structure of database objects such as tables and indexes. Commands: `CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `RENAME`.
>
> **DML (Data Manipulation Language)** is used to insert, change, delete and read the data inside tables. Commands: `INSERT`, `UPDATE`, `DELETE`, `SELECT`.
>
> Key difference: DDL works on the **schema (structure)**; DML works on the **data (rows)**.

3. The 10-Mark Answer (full write-up)

3.1 What is DDL?

DDL stands for **Data Definition Language**. It is the part of SQL that **defines** the schema. The schema is the blueprint of the database: table names, column names, data types and rules. DDL commands are usually **auto-committed** in systems like Oracle and MySQL, which means the change is saved permanently the moment the command runs.

Command	What it does	Example
CREATE	Makes a new table, view, index or database	`CREATE TABLE Student (RollNo INT, Name VARCHAR(50));`
ALTER	Changes an existing table (add, modify or drop a column)	`ALTER TABLE Student ADD Email VARCHAR(80);`

Command	What it does	Example
DROP	Deletes the whole table (structure and data) permanently	`DROP TABLE Student;`
TRUNCATE	Removes **all rows** quickly but keeps the empty table structure	`TRUNCATE TABLE Student;`
RENAME	Changes the name of a table	`RENAME TABLE Student TO Pupil;`

3.2 What is DML?

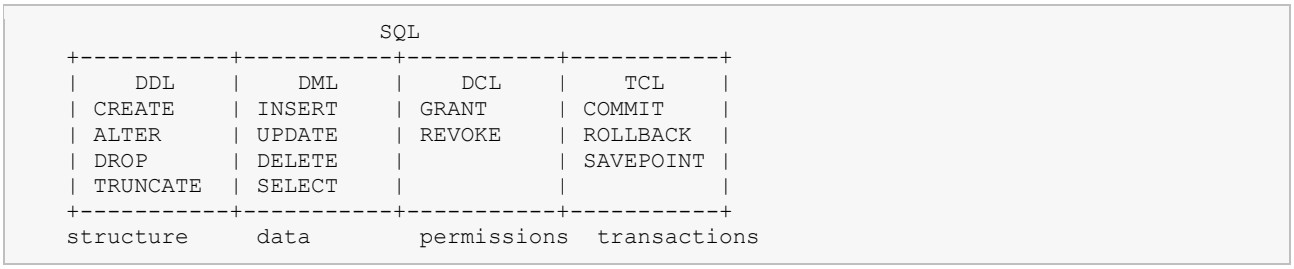
DML stands for **Data Manipulation Language**. It is the part of SQL used to **work with the data** inside tables. DML commands are **not auto-committed**: you can undo them with `ROLLBACK` or save them with `COMMIT`.

Command	What it does	Example
INSERT	Adds a new row	`INSERT INTO Student VALUES (1, 'Asif', 8.1);`
UPDATE	Changes existing rows	`UPDATE Student SET CGPA = 8.5 WHERE RollNo = 1;`
DELETE	Removes selected rows (table stays)	`DELETE FROM Student WHERE RollNo = 1;`
SELECT	Reads / retrieves rows (some books call this DQL)	`SELECT * FROM Student;`

3.3 Differences between DDL and DML (the table examiners want)

Basis	DDL	DML
Full form	Data Definition Language	Data Manipulation Language
Purpose	Defines or changes the **structure (schema)**	Inserts, changes, deletes, retrieves **data**
Works on	Tables, views, indexes, schemas	Rows (records) inside tables
Commands	CREATE, ALTER, DROP, TRUNCATE, RENAME	INSERT, UPDATE, DELETE, SELECT
Commit	**Auto-commit** (cannot normally be rolled back)	Needs manual COMMIT; **can be rolled back**
WHERE clause	Not used	Used to choose which rows to affect
Effect	Changes how data is stored	Changes what data is stored
Speed on big data	TRUNCATE is very fast (no row-by-row work)	DELETE is slower (row by row, logs each row)
Used by	DBA, database designer	Application developers and end users

3.4 Other SQL sub-languages (bonus marks)



3.5 DELETE vs TRUNCATE vs DROP (very common follow-up)

	DELETE	TRUNCATE	DROP
Type	DML	DDL	DDL
Removes	Chosen rows (with WHERE) or all rows	All rows only	Whole table: data + structure
WHERE allowed?	Yes	No	No
Rollback?	Yes	Usually no	Usually no
Table remains?	Yes	Yes (empty)	No

4. Real-Life Use

In real software projects

DDL is run through **migration tools** (Flyway, Prisma Migrate, Liquibase) when a new version is deployed. DML runs from the **application code** on every user click: signing up (`INSERT`), changing a profile (`UPDATE`), viewing orders (`SELECT`). A wrong `DROP` is permanent, so developers review DDL very carefully.

5. Exam Tips

Remember

- Always write the **table of differences** (at least 6 rows) and give **one example of each**.
- Mention "auto-commit" for DDL. It is the point most students forget.
- Mnemonic: **D**efinition = **D**esign. **M**anipulation = **M**oving data.

Question 2: Trigger

> **The question (typos fixed)**
>
> What is a trigger?
>
> **Unit:** Advanced SQL / PL-SQL

1. Understand It First (Plain English)

> **Think of an automatic light**
>
> A motion-sensor light switches on by itself when you walk past. You never press a button. A **trigger** is the same idea in a database: a small piece of code that runs **automatically** when something happens to a table (a row is inserted, updated or deleted).
>
> You do not call a trigger. The database calls it for you.

2. The 2-Mark Answer (write this)

> **2-Mark Answer**
>
> A **trigger** is a stored program that is **executed automatically** by the DBMS when a specified event (`INSERT`, `UPDATE` or `DELETE`) occurs on a table.
>
> It is used to enforce business rules, maintain audit logs and keep data consistent.

3. The 10-Mark Answer (full write-up)

3.1 Definition

A trigger is a special **stored database object** that fires (runs) automatically in response to a data change event. It follows the **ECA model**:

- **E: Event.** The change that wakes the trigger (INSERT, UPDATE or DELETE).
- **C: Condition.** An optional check (for example "only if salary changed").
- **A: Action.** The code that runs if the condition is true.

3.2 Types of triggers

Based on	Types	Meaning
Timing	BEFORE / AFTER	BEFORE runs before the change is made (good for validation). AFTER runs after it (good for audit/logging).
Event	INSERT / UPDATE / DELETE	The operation that fires the trigger
Level	Row-level / Statement-level	Row-level fires once **for each row** affected. Statement-level fires **once per statement**.

3.3 OLD and NEW (important idea)

Inside a row-level trigger, `OLD` holds the values of the row **before** the change and `NEW` holds the values **after** the change.

- INSERT trigger: only `NEW` exists (there is no old row).
- DELETE trigger: only `OLD` exists (there is no new row).
- UPDATE trigger: both `OLD` and `NEW` exist.

3.4 General syntax

```
CREATE TRIGGER trigger_name
{ BEFORE | AFTER } { INSERT | UPDATE | DELETE }
ON table_name
FOR EACH ROW
BEGIN
    -- statements to execute
END;
```

3.5 Example 1: audit log of salary changes (AFTER UPDATE)

Goal: whenever an employee's salary changes, save the old and new salary in a log table.

```
CREATE TABLE Salary_Audit (
    emp_id INT, old_salary INT, new_salary INT, changed_at DATETIME
);

DELIMITER //
CREATE TRIGGER trg_salary_audit
AFTER UPDATE ON Employee
FOR EACH ROW
BEGIN
    IF OLD.salary <> NEW.salary THEN
        INSERT INTO Salary_Audit
        VALUES (OLD.emp_id, OLD.salary, NEW.salary, NOW());
    END IF;
END//
DELIMITER ;
```

3.6 Example 2: validation (BEFORE INSERT)

Goal: stop anyone inserting a negative salary.

```
CREATE TRIGGER trg_check_salary
BEFORE INSERT ON Employee
FOR EACH ROW
BEGIN
    IF NEW.salary < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary cannot be negative';
    END IF;
END;
```

3.7 How it works (diagram)

```
  UPDATE Employee SET salary = 60000 WHERE emp_id = 7;
                      |
                      v
           +--------------------+
           |   Employee table   |   <-- EVENT happens
           +---------+----------+
```

```
                | fires
                v
          +---------------------+
          |  CONDITION check    |-- false --> do nothing
          | OLD.sal <> NEW.sal? |
          +---------+-----------+
                    | true
                    v
          +---------------------+
          |  ACTION: insert     |
          |  row in Salary_Audit
          +---------------------+
```

3.8 Advantages and disadvantages

Advantages	Disadvantages
Automatic: rules are enforced without anyone remembering to run them	Hard to debug: it runs invisibly in the background
Central: every application gets the same rule	Slows down INSERT/UPDATE/DELETE (extra work each time)
Good for audit trails and complex integrity checks	One trigger can fire another (cascading), causing confusing behaviour
Reduces duplicate code in applications	Harder to move to another DBMS (syntax differs)

To remove a trigger: `DROP TRIGGER trg_salary_audit;`

4. Real-Life Use

In real software projects

- **Banking:** every balance change on an `accounts` table is copied by a trigger into a ledger table. Regulators require this history.
- **E-commerce:** an `AFTER INSERT` trigger on `order_items` reduces `inventory.stock`.
- Many teams keep business logic in application code and use triggers only for audit logs and rules that must hold for **every** writer.

5. Exam Tips

Remember

- Write the word **ECA (Event-Condition-Action)**. It earns marks.
- Always include **one syntax block and one example**.
- Mention BEFORE/AFTER and Row/Statement levels.

Question 3: Advantages of Normalization

> **The question (typos fixed)**
>
> What are the advantages of normalization?
>
> **Unit:** Database Design and Normalization

1. Understand It First (Plain English)

> **The messy phone book**
>
> Imagine your friend Ravi has two phone numbers, and you wrote his address next to **every** entry in your notebook. When Ravi moves house, you must find and fix every single entry. If you miss one, your notebook now disagrees with itself.
>
> **Normalization** means organising data so each fact is written **only once**. Then changing it means changing it in **one place**.
>
> The enemy of normalization is **redundancy** (the same data repeated) and the problems it causes, called **anomalies**.

2. The 2-Mark Answer (write this)

> **2-Mark Answer**
>
> **Normalization** is the process of organising data into well-structured tables to remove redundancy and anomalies.
>
> Advantages: (1) reduces data redundancy, (2) removes insertion, deletion and update anomalies, (3) improves data integrity and consistency, (4) saves storage space, (5) makes the database easier to maintain.

3. The 10-Mark Answer (full write-up)

3.1 Definition

Normalization is the process of **decomposing (splitting)** a large table into smaller, well-organised tables using **functional dependencies**, so that every fact is stored in exactly one place. It was introduced by E. F. Codd. The stages are called normal forms: 1NF, 2NF, 3NF, BCNF and so on.

3.2 The problem normalization solves

Look at this single table that stores employees and their departments together:

EmpID	EmpName	DeptID	DeptName	DeptLocation
1	Ravi	D1	CSE	Block A
2	Sita	D1	CSE	Block A

EmpID	EmpName	DeptID	DeptName	DeptLocation
3	Amit	D2	ECE	Block B

"CSE / Block A" is written twice. This repetition is **redundancy**, and it causes three **anomalies** (problems):

- **Insertion anomaly:** You cannot add a new department (say, "Mech") until at least one employee exists in it, because the employee columns would be empty.
- **Deletion anomaly:** If Amit (the only ECE employee) leaves and you delete his row, you also lose the fact that ECE exists in Block B.
- **Update anomaly:** If CSE moves to Block C, you must edit every CSE row. If you miss one, the data becomes inconsistent (some rows say Block A, others Block C).

3.3 After normalization

Split into two tables. Each fact now lives in one place:

EmpID	EmpName	DeptID (FK)
1	Ravi	D1
2	Sita	D1
3	Amit	D2

DeptID (PK)	DeptName	DeptLocation
D1	CSE	Block A
D2	ECE	Block B

Now CSE moving to Block C is **one** update. A new department can be added with no employees. Deleting Amit does not delete ECE.

3.4 Advantages of normalization

1. **Minimises redundancy.** Each fact is stored once, so storage is saved.
2. **Removes anomalies.** Insertion, deletion and update anomalies disappear (explained above).
3. **Improves data integrity and consistency.** There is a single source of truth, so the database never contradicts itself.
4. **Better organisation.** Each table represents one real-world thing (student, department, course), so the design is clean and logical.
5. **Easier maintenance.** A change in the structure or a fact affects fewer places.
6. **Faster updates.** Narrow tables mean less data to write for each change.
7. **Smaller, faster indexes** on narrow tables.
8. **Flexible querying.** Because data is split sensibly, you can combine it in many ways with joins.
9. **Better concurrency.** Different users update different small tables, so there are fewer lock conflicts.
10. **Easier to extend.** New attributes or entities can be added without disturbing existing tables.

3.5 Disadvantages (examiners sometimes ask)

- More tables means more **JOINs**, which can make reading (SELECT) slower.
- Queries become more complex to write.
- Over-normalising makes the design hard to understand. Some systems deliberately **denormalise** (add controlled redundancy) to make reads faster.

4. Real-Life Use

> **In real software projects**
>
> In an online shop, a customer's address is stored in one `addresses` table, not copied into every `orders` row. When the customer changes address, **one row** is updated. Transaction systems (OLTP) are normally designed up to **3NF**. Reporting databases (data warehouses) are often denormalised on purpose for fast reads.

5. Exam Tips

> **Remember**
>
> - Draw the **before and after** tables. This alone earns half the marks.
> - Name all three anomalies with a one-line example each.
> - Finish with one disadvantage to show depth.

Question 4: Types of Functional Dependencies

> **The question (typos fixed)**
>
> What are the various types of functional dependencies?
>
> **Unit:** Relational Database Design

1. Understand It First (Plain English)

What is a functional dependency?

A **functional dependency (FD)** means "if you know this, you can find out that". It is written `X → Y` and read "X determines Y".

Example: your **Roll Number** tells us your **Name** (no two students share a roll number). So `RollNo → Name`. The reverse is not true: a name can be shared by two students.

The left side (`X`) is called the **determinant**. The right side (`Y`) is the **dependent**.

A **key** (candidate key) is the smallest set of columns that can uniquely identify a row. A column that is part of some key is called a **prime attribute**; any other column is **non-prime**. You will need these words below.

2. The 2-Mark Answer (write this)

2-Mark Answer

A **functional dependency** `X → Y` means each value of X is linked to exactly one value of Y.

Types: **trivial**, **non-trivial**, **completely non-trivial**, **full**, **partial**, **transitive**, **multivalued**, and **join** dependency.

3. The 10-Mark Answer (full write-up)

3.1 Formal definition

For a relation R with attribute sets X and Y, the FD `X → Y` holds if, for any two rows t1 and t2: **whenever t1[X] = t2[X], then t1[Y] = t2[Y]**. In words: two rows that agree on X must also agree on Y.

We use this table in the examples:

RollNo	Name	CourseID	CourseName	Grade
1	Ravi	C1	DBMS	A
1	Ravi	C2	OS	B
2	Sita	C1	DBMS	B

Here the key is `{RollNo, CourseID}` (a student takes many courses, so you need both to identify a row).

3.2 Type 1: Trivial dependency

`X → Y` is **trivial** when **Y is a subset of X** (the right side is already contained in the left side). It is always true, so it tells us nothing new.

Example: `{RollNo, Name} → RollNo`. Also `A → A`.

3.3 Type 2: Non-trivial dependency

`X → Y` is **non-trivial** when **Y is not a subset of X**. This is a real, useful dependency.

Example: `RollNo → Name`.

3.4 Type 3: Completely non-trivial dependency

`X → Y` where **X and Y have nothing in common** ($X \cap Y = \emptyset$).

Example: `RollNo → Name` is completely non-trivial. But a dependency like `{A, B} → {B, C}` is only non-trivial, not completely non-trivial, because B appears on both sides.

3.5 Type 4: Full functional dependency

Y depends on **the whole** of X and not on any smaller part of X. If you remove any attribute from X, the dependency breaks.

Example: `{RollNo, CourseID} → Grade`. Neither RollNo alone nor CourseID alone decides the grade, so you need both. This is a full dependency.

3.6 Type 5: Partial dependency

A **non-prime** attribute depends on only **a part** of a composite key.

Example: the key is `{RollNo, CourseID}` but `RollNo → Name`. Name depends only on RollNo, which is only part of the key. **This is the problem that 2NF removes.**

3.7 Type 6: Transitive dependency

If `X → Y` and `Y → Z` (and Y does not determine X), then `X → Z` is a **transitive** dependency. It works indirectly, through a middle attribute.

Example: Student(RollNo, Name, DeptID, DeptName). `RollNo → DeptID` and `DeptID → DeptName`, so `RollNo → DeptName` is transitive. **This is the problem that 3NF removes.**

3.8 Type 7: Multivalued dependency (MVD)

Written `X →→ Y`. For one value of X there is a **set** of Y values, and this set does not depend on the other attributes. It appears when two independent multi-valued facts are stored together. Removed by **4NF**.

Person	Phone	Hobby
Ravi	98765	Chess
Ravi	98765	Music
Ravi	91234	Chess
Ravi	91234	Music

`Person →→ Phone` and `Person →→ Hobby`. Phones and hobbies are independent, yet every combination must be stored: this wastes space.

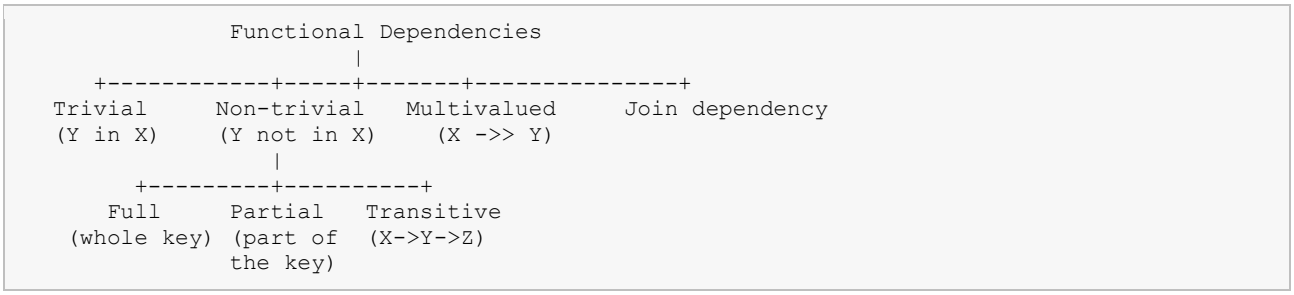
3.9 Type 8: Join dependency

A relation can be rebuilt exactly by joining several of its projections (smaller tables). Removed by **5NF**. (Rarely asked in detail.)

3.10 Armstrong's axioms (rules to derive new FDs)

Rule	Statement	Example
Reflexivity	If Y ⊆ X then X → Y	{A,B} → A
Augmentation	If X → Y then XZ → YZ	A → B gives AC → BC
Transitivity	If X → Y and Y → Z then X → Z	A → B, B → C gives A → C
Union (derived)	If X → Y and X → Z then X → YZ	A → B, A → C gives A → BC
Decomposition (derived)	If X → YZ then X → Y and X → Z	A → BC gives A → B, A → C
Pseudo-transitivity (derived)	If X → Y and WY → Z then WX → Z	A → B, CB → D gives CA → D

3.11 Summary diagram



4. Real-Life Use

> **In real software projects**
>
> Finding FDs is the first step of schema design. Developers say "an order id determines the customer and order date" and "a product id determines its price". Each such group of dependent facts becomes its own table. Partial and transitive dependencies are the warning signs that a table needs splitting.

5. Exam Tips

> **Remember**
>
> - Give the **definition, then each type with a one-line example**. Use the same table throughout.
> - The "problem each type causes" line is a favourite: **partial = 2NF problem**, **transitive = 3NF problem**, **MVD = 4NF problem**.
> - Writing Armstrong's axioms shows depth and often earns the last 2 marks.

Question 5: Constraints

> **The question (typos fixed)**
>
> What are constraints?
>
> **Unit:** Relational Model and SQL: Integrity Constraints

1. Understand It First (Plain English)

> **Rules at the door of a club**
>
> A club has a doorman with rules: "Age 18 or above", "No duplicate entry", "Must have a ticket". Anyone who breaks a rule is refused.
>
> **Constraints** are the doorman of a table. They are rules attached to columns. If you try to insert or change data in a way that breaks a rule, the database **rejects** it with an error. This keeps the data correct.

2. The 2-Mark Answer (write this)

> **2-Mark Answer**
>
> **Constraints** are rules applied to table columns to ensure the **accuracy, validity and integrity** of data. The DBMS rejects any operation that violates them.
>
> Types: `NOT NULL`, `UNIQUE`, `PRIMARY KEY`, `FOREIGN KEY`, `CHECK`, `DEFAULT`.

3. The 10-Mark Answer (full write-up)

3.1 Definition

A constraint is a condition that **every row** of a table must satisfy. Constraints are declared when the table is created (`CREATE TABLE`) or later (`ALTER TABLE`). They are the way a relational database enforces **integrity** (correctness and consistency of data).

3.2 Types of SQL constraints

Constraint	Meaning	Allows NULL?	Allows duplicates?
NOT NULL	Column can never be empty	No	Yes
UNIQUE	All values in the column must be different	Yes	No
PRIMARY KEY	Unique + Not Null. Identifies each row. Only **one** per table	No	No
FOREIGN KEY	Value must exist in the primary key of another (parent) table, or be NULL	Yes	Yes

Constraint	Meaning	Allows NULL?	Allows duplicates?
CHECK	Value must satisfy a condition, for example Age >= 17	Yes	Yes
DEFAULT	Gives a value automatically if none is supplied	n/a	n/a

(NULL means "unknown or missing". It is not zero and not an empty string.)

3.3 Keys (needed to understand PRIMARY and FOREIGN KEY)

Key	Meaning
Super key	Any set of columns that uniquely identifies a row (may contain extra columns)
Candidate key	A **minimal** super key (remove any column and it stops being unique). A table can have several.
Primary key	The candidate key chosen by the designer to identify rows. Cannot be NULL.
Alternate key	Candidate keys that were **not** chosen as the primary key
Composite key	A key made of two or more columns
Foreign key	A column that refers to the primary key of another table. It creates the link between tables.

3.4 Categories of integrity (theory view)

- **Domain integrity:** values must come from the allowed set (data type, CHECK, NOT NULL, DEFAULT).
- **Entity integrity:** the primary key can never be NULL and must be unique, so every row is identifiable.
- **Referential integrity:** a foreign key value must match an existing primary key in the parent table (or be NULL). No "orphan" rows.
- **Key constraint:** no two rows can have the same key value.

3.5 SQL example with all constraints

```
CREATE TABLE Department (
    DeptID    INT PRIMARY KEY,
    DeptName  VARCHAR(40) NOT NULL UNIQUE
);

CREATE TABLE Student (
    RollNo   INT PRIMARY KEY,
    Name     VARCHAR(50) NOT NULL,
    Email    VARCHAR(80) UNIQUE,
    Age      INT CHECK (Age >= 17),
    Country  VARCHAR(30) DEFAULT 'India',
    DeptID   INT,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
        ON DELETE CASCADE
);
```

Adding a constraint to an existing table: `ALTER TABLE Student ADD CONSTRAINT chk_age CHECK (Age >= 17);`

3.6 Referential actions (what happens when the parent row is deleted)

Option	Effect
ON DELETE CASCADE	Child rows are deleted automatically with the parent

Option	Effect
ON DELETE SET NULL	Child's foreign key is set to NULL
ON DELETE RESTRICT / NO ACTION	Deleting the parent is **blocked** while children exist

3.7 Foreign key diagram

```
   Department (PARENT)              Student (CHILD)
  +--------+----------+          +--------+------+--------+
  | DeptID | DeptName |          | RollNo | Name | DeptID |  <- FK
  |  (PK)  |          |          +--------+------+--------+
  |  D1    | CSE      | <------- |   1    | Ravi |  D1    |  OK
  |  D2    | ECE      | <------- |   3    | Amit |  D2    |  OK
  +--------+----------+          |   2    | Sita |  D9    |  REJECTED
                                 +--------+------+--------+
                                 (D9 does not exist in Department)
```

4. Real-Life Use

> **In real software projects**
>
> Constraints are the **last line of defence against bugs**. Application checks can fail or be bypassed, but a `UNIQUE(email)` constraint guarantees that no two accounts share an email even when two users sign up at the same instant. A `CHECK (balance >= 0)` stops a bank balance going negative.

5. Exam Tips

> **Remember**
>
> - Write the table of six constraints, then the **CREATE TABLE example**. The example is where marks are.
> - Mention the **3 integrity types** (domain, entity, referential).
> - Know the difference: **PRIMARY KEY = UNIQUE + NOT NULL**, and only one per table.

Question 6: Relational Algebra

> **The question (typos fixed)**
>
> What is relational algebra? Explain the different operations of relational algebra with examples.
>
> **Unit:** Relational Model and Query Languages

1. Understand It First (Plain English)

> **A calculator for tables**
>
> A normal calculator takes numbers and gives a number. **Relational algebra** is a "calculator" that takes **tables** and gives a **table**. Each operator does one small job: pick some rows, pick some columns, join two tables, and so on.
>
> It is **procedural**: you describe the steps (how to get the answer). Because every result is again a table, you can chain operators together. SQL is built on top of these ideas.
>
> Words: **relation** = table, **tuple** = row, **attribute** = column.

2. The 2-Mark Answer (write this)

> **2-Mark Answer**
>
> **Relational algebra** is a procedural query language. It has a set of operators that take one or two relations (tables) as input and produce a new relation as output.
>
> **Basic operators:** Select (σ), Project (π), Union (∪), Set difference (−), Cartesian product (×), Rename (ρ).
>
> **Derived operators:** Intersection (∩), Join (⋈), Division (÷).

3. The 10-Mark Answer (full write-up)

3.1 Tables used in all examples

STUDENT

RollNo	Name	Dept	CGPA
1	Ravi	CSE	8.5
2	Sita	ECE	7.9
3	Amit	CSE	9.1

ENROLL

RollNo	CourseID
1	C1

RollNo	CourseID
1	C2
3	C1

3.2 A. Fundamental (basic) operations

1. Selection (σ): picks ROWS that satisfy a condition.

Syntax: `σ condition (Relation)`. Example: `σ Dept = 'CSE' (STUDENT)`

RollNo	Name	Dept	CGPA
1	Ravi	CSE	8.5
3	Amit	CSE	9.1

Conditions can be combined with ∧ (AND), ∨ (OR), ¬ (NOT) and compared with =, ≠, <, >, ≤, ≥.

2. Projection (π): picks COLUMNS and removes duplicates.

Syntax: `π column list (Relation)`. Example: `π Dept (STUDENT)` gives {CSE, ECE}. CSE appears once because duplicates are removed (relations are sets).

Dept
CSE
ECE

3. Union (∪): all rows that are in R or in S (or both).

Both relations must be **union compatible**: same number of columns and matching data types. Example: `π RollNo (STUDENT) ∪ π RollNo (ENROLL)` = {1, 2, 3}.

4. Set difference (−): rows in R but not in S.

Example: `π RollNo (STUDENT) − π RollNo (ENROLL)` = {2}. This answers "students who have not enrolled in any course" (Sita). Order matters: R − S is not the same as S − R.

5. Cartesian product (×): pairs every row of R with every row of S.

If R has m rows and S has n rows, R × S has **m × n** rows and the columns of both. Small example: A(x) = {1, 2} and B(y) = {a, b}:

x	y
1	a
1	b
2	a
2	b

STUDENT × ENROLL would have 3 × 3 = 9 rows, mostly meaningless pairs. That is why we usually follow a product by a selection, which is exactly what a join is.

6. Rename (ρ): gives a new name to a relation or its columns.

`ρ S (STUDENT)` renames the relation to S. `ρ S(ID, Name, Dept, CGPA) (STUDENT)` renames the columns too. Needed when a table is joined with itself.

3.3 B. Derived (additional) operations

These can be written using the basic ones, but are used so often they have their own symbols.

7. Intersection (∩): rows common to both. `R ∩ S = R − (R − S)`.

Example: `π RollNo (STUDENT) ∩ π RollNo (ENROLL)` = {1, 3}.

8. Join (⋈): a Cartesian product followed by a selection.

- **Theta join (⋈θ):** join using any condition θ such as `<`, `>`, `=`.
- **Equi join:** theta join where the condition uses only `=`.
- **Natural join (⋈):** equi join on all columns with the same name, keeping one copy of the common column.

`STUDENT ⋈ ENROLL` (matches on RollNo). Sita has no enrollment, so she disappears:

RollNo	Name	Dept	CGPA	CourseID
1	Ravi	CSE	8.5	C1
1	Ravi	CSE	8.5	C2
3	Amit	CSE	9.1	C1

Outer joins keep unmatched rows and fill the gaps with NULL:

- **Left outer join (⟕):** keeps all rows of the left table. Sita appears with CourseID = NULL.
- **Right outer join (⟖):** keeps all rows of the right table.
- **Full outer join (⟗):** keeps unmatched rows from both sides.

9. Division (÷): finds rows related to ALL rows of another table.

Typical question: "Find students enrolled in **all** courses." Let COURSES = {C1, C2}. Then `ENROLL ÷ COURSES` gives {1}, because only RollNo 1 is enrolled in both C1 and C2. (Amit has only C1.)

10. Aggregation and grouping (extended operation):

`Dept 𝒢 COUNT(RollNo), AVG(CGPA) (STUDENT)` groups by Dept and gives:

Dept	COUNT	AVG(CGPA)
CSE	2	8.8
ECE	1	7.9

3.4 Combined query example

Query: names of CSE students enrolled in course C1.

`π Name ( σ Dept='CSE' ∧ CourseID='C1' ( STUDENT ⋈ ENROLL ) )` gives **Ravi and Amit**.

Read it from the inside out: join the tables, pick the rows with CSE and C1, then keep only the Name column.

3.5 Query tree

```
Query tree for  pi Name ( sigma Dept='CSE' (STUDENT) )

          pi Name          <- last step: keep the Name column
             |
       sigma Dept='CSE'    <- second step: keep only CSE rows
             |
          STUDENT          <- first: read the table
```

3.6 Summary of operators

Symbol	Name	Kind	Works on	SQL equivalent
σ	Selection	Basic, unary	Rows	WHERE
π	Projection	Basic, unary	Columns	SELECT col
∪	Union	Basic, binary	Two relations	UNION
−	Set difference	Basic, binary	Two relations	EXCEPT / MINUS
×	Cartesian product	Basic, binary	Two relations	CROSS JOIN
ρ	Rename	Basic, unary	Names	AS
∩	Intersection	Derived	Two relations	INTERSECT
⋈	Join	Derived	Two relations	JOIN
÷	Division	Derived	Two relations	No direct keyword

4. Real-Life Use

> **In real software projects**
>
> When you run an SQL query, the database converts it into a relational algebra tree and then **rearranges it to run faster**. For example, it pushes the selection (σ) down so that rows are filtered **before** a join. The `EXPLAIN` command shows this plan as a tree of Scan, Filter and Join steps.

5. Exam Tips

> **Remember**
>
> - Start with the definition, then show **sample tables** and a **result table for every operator**. Examiners reward worked examples.
> - Mention **union compatibility** and that projection removes duplicates.
> - Finish with the summary table of symbols.

Question 7: ACID Properties

> **The question (typos fixed)**
>
> Explain the ACID properties.
>
> **Unit:** Transaction Processing

1. Understand It First (Plain English)

> **The ATM promise**
>
> You transfer Rs 1000 from your account to a friend. Inside the bank, two steps happen: **take 1000 from you**, then **give 1000 to your friend**. A **transaction** is a group of steps that must be treated as **one unit of work**.
>
> What if the power fails after step 1? You would lose money. What if two transfers happen at once and confuse each other? The **ACID** properties are four promises the database makes so this never goes wrong.

2. The 2-Mark Answer (write this)

> **2-Mark Answer**
>
> ACID are the four properties that guarantee reliable transactions:
>
> - **Atomicity:** all operations happen, or none do.
> - **Consistency:** the database moves from one valid state to another valid state.
> - **Isolation:** concurrent transactions do not interfere with each other.
> - **Durability:** once committed, changes are permanent even after a crash.

3. The 10-Mark Answer (full write-up)

3.1 What is a transaction?

A **transaction** is a logical unit of work made of one or more database operations (reads and writes) that must be executed as a whole. It ends with `COMMIT` (save) or `ROLLBACK` (undo).

Running example: transfer Rs 1000 from account A (balance 5000) to account B (balance 2000).

```
T:  1. READ(A)
    2. A = A - 1000
    3. WRITE(A)
    4. READ(B)
    5. B = B + 1000
    6. WRITE(B)
    7. COMMIT
```

3.2 Atomicity (all or nothing)

The transaction either executes **completely** or **not at all**. If the system crashes after step 3 (A is debited but B is not credited), the DBMS must **undo** step 3 so the money is not lost.

Who ensures it: the Recovery Manager, using the **undo log** and rollback.

```
Start:           A = 5000   B = 2000    (total 7000)
After W(A):      A = 4000   B = 2000    <-- crash here!
After rollback:  A = 5000   B = 2000    atomicity kept
If all succeed:  A = 4000   B = 3000    COMMIT
```

3.3 Consistency (valid state to valid state)

A transaction must take the database from one **consistent state** (all rules and constraints true) to another consistent state. In the example the rule "A + B = 7000" is true before and after the transfer.

Who ensures it: the programmer who writes correct transactions plus the database constraints (primary key, foreign key, CHECK). Atomicity and isolation support it.

3.4 Isolation (no interference)

Even when many transactions run at the same time, the result must be the same as if they ran **one after another**. Without isolation, another transaction reading A and B between steps 3 and 6 would see a total of 6000, which is wrong.

Who ensures it: the Concurrency Control Manager, using locking (two-phase locking), timestamps or multi-version concurrency control (MVCC).

Problems that appear without isolation:

- **Dirty read:** reading data written by a transaction that has not committed yet (and may roll back).
- **Non-repeatable read:** reading the same row twice in one transaction and getting different values because someone changed it in between.
- **Phantom read:** running the same query twice and getting extra rows because someone inserted new ones.
- **Lost update:** two transactions overwrite each other's changes.

SQL isolation levels (weakest to strongest):

Level	Dirty read	Non-repeatable read	Phantom read
Read Uncommitted	Possible	Possible	Possible
Read Committed	Prevented	Possible	Possible
Repeatable Read	Prevented	Prevented	Possible
Serializable	Prevented	Prevented	Prevented

3.5 Durability (permanent after commit)

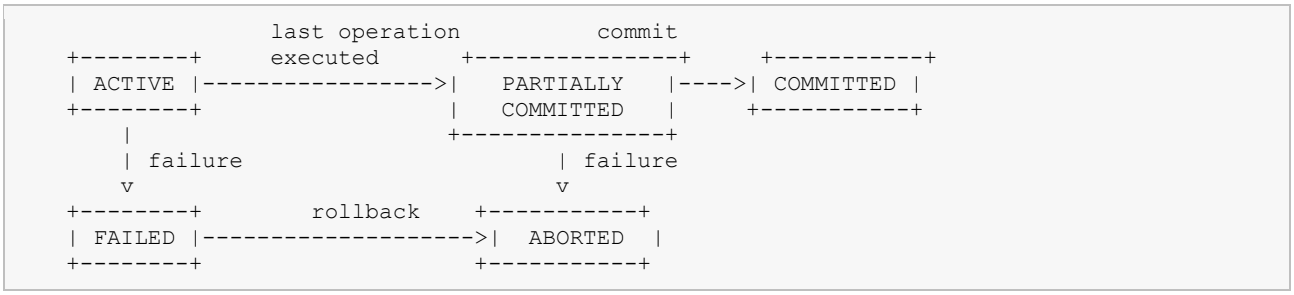
Once the database says "COMMIT successful", the changes are **permanent**, even if the power fails one second later.

Who ensures it: the Recovery Manager, using **write-ahead logging (WAL)**: the log is written to disk **before** the commit is confirmed, so after a crash the database can **redo** the committed changes.

3.6 Summary table

Property	Promise	Handled by
Atomicity	All or nothing	Recovery manager (undo log, rollback)
Consistency	Valid state to valid state	Constraints and application logic
Isolation	Transactions do not interfere	Concurrency control manager (locks, MVCC)
Durability	Committed data is permanent	Recovery manager (redo log, WAL)

3.7 Transaction state diagram



States: Active (executing), Partially committed (last statement done, not yet saved), Committed (saved permanently), Failed (cannot continue), Aborted (rolled back; the system may restart it or cancel it).

4. Real-Life Use

> **In real software projects**
>
> - **Payments:** debit and credit must be atomic and durable.
> - **E-commerce checkout:** "create order + reduce stock + record payment" is one transaction. If payment fails, the stock reduction is rolled back.
> - In SQL this looks like `BEGIN; ... COMMIT;` or `ROLLBACK;`.

5. Exam Tips

> **Remember**
>
> - Use the **bank transfer example** for all four properties. It is the most expected answer.
> - Write **who ensures each property** (recovery manager, concurrency manager). Few students do.
> - Mnemonic: **A**ll-or-nothing, **C**orrect state, **I**ndependent, **D**urable.

Question 8: Tuple, Relational Calculus, TRC and DRC

> **The question (typos fixed)**
>
> What are tuple, relational calculus, tuple relational calculus and domain relational calculus?
>
> **Unit:** Relational Query Languages

1. Understand It First (Plain English)

> **Two ways to ask for a sandwich**
>
> **Relational algebra** is like giving a recipe: "take bread, add cheese, toast it". It says **how**.
>
> **Relational calculus** is like saying "I want a toasted cheese sandwich". It says **what** you want and lets the system work out how. It is **non-procedural (declarative)**.
>
> There are two flavours. In **TRC** your variable stands for a whole **row**. In **DRC** your variable stands for a single **column value**.

2. The 2-Mark Answer (write this)

> **2-Mark Answer**
>
> - **Tuple:** one row (record) of a relation.
> - **Relational calculus:** a non-procedural query language based on predicate logic. It specifies **what** to retrieve, not how.
> - **TRC (Tuple Relational Calculus):** variables represent tuples. Form: `{ t | P(t) }`.
> - **DRC (Domain Relational Calculus):** variables represent attribute values. Form: `{ <x1, x2, ..., xn> | P(x1, ..., xn) }`.

3. The 10-Mark Answer (full write-up)

3.1 Tuple

A **tuple** is a single row of a table: an ordered list of values, one per attribute. For example `(1, Ravi, CSE, 8.5)` is a tuple of STUDENT.

Tables used below:

RollNo	Name	Dept	CGPA
1	Ravi	CSE	8.5
2	Sita	ECE	7.9
3	Amit	CSE	9.1

ENROLL(RollNo, CourseID) has rows (1, C1), (1, C2), (3, C1).

3.2 Relational calculus

Relational calculus describes the result using a **logical condition (predicate)**. It has the **same expressive power** as relational algebra, provided the query is **safe** (gives a finite answer). Symbols used:

Symbol	Read as
∧	AND
∨	OR
¬	NOT
⇒	implies (if ... then ...)
∃	there exists (at least one)
∀	for all (every)
∈	belongs to (is a row of)

3.3 Tuple Relational Calculus (TRC)

General form: `{ t | P(t) }`. It means "the set of all tuples t for which the condition P is true".

- **Tuple variable:** stands for a whole row. `t ∈ STUDENT` (or written `STUDENT(t)`) says t is a row of STUDENT.
- **Attribute access:** `t.CGPA` means the CGPA value of row t.
- **Free variable:** appears in the answer. **Bound variable:** controlled by ∃ or ∀.

Examples:

1. All CSE students: `{ t | t ∈ STUDENT ∧ t.Dept = 'CSE' }`
2. Names of students with CGPA above 8: `{ t.Name | t ∈ STUDENT ∧ t.CGPA > 8 }`
3. Students enrolled in at least one course: `{ s | s ∈ STUDENT ∧ ∃e ( e ∈ ENROLL ∧ e.RollNo = s.RollNo ) }`. Answer: Ravi, Amit.
4. Students enrolled in **no** course: `{ s | s ∈ STUDENT ∧ ¬∃e ( e ∈ ENROLL ∧ e.RollNo = s.RollNo ) }`. Answer: Sita.

3.4 Domain Relational Calculus (DRC)

General form: `{ <x1, x2, ..., xn> | P(x1, x2, ..., xn) }`. Here each variable stands for **one column value** (a member of a domain), not a whole row. You list a variable for each column in the order they appear.

Examples:

1. All CSE students: `{ <r, n, d, c> | <r, n, d, c> ∈ STUDENT ∧ d = 'CSE' }`
2. Names of students with CGPA above 8: `{ <n> | ∃r, d, c ( <r, n, d, c> ∈ STUDENT ∧ c > 8 ) }`
3. Names of students enrolled in C1: `{ <n> | ∃r, d, c ( <r, n, d, c> ∈ STUDENT ∧ <r, 'C1'> ∈ ENROLL ) }`. Answer: Ravi, Amit.

3.5 Difference between TRC and DRC

Basis	TRC	DRC
Variable stands for	A whole tuple (row)	A single attribute value
Notation	`{ t | P(t) }`	`{ <x1,...,xn> | P(x1,...,xn) }`
Access to attribute	`t.attribute`	By position in the tuple
Number of variables	Fewer (one per row)	More (one per column)
Practical language based on it	**SQL**	**QBE** (Query By Example)
Length of queries	Shorter	Longer

3.6 Safe expressions

A query is **safe** if it always returns a finite set that uses only values present in the database. For example `{ t | ¬( t ∈ STUDENT ) }` is **unsafe**, because infinitely many tuples are not in STUDENT. Only safe queries are allowed.

3.7 Relational algebra vs relational calculus

Relational algebra	Relational calculus
Procedural: says **how** to get the result	Declarative: says **what** result is wanted
Uses operators (σ, π, ⋈ ...)	Uses predicate logic (∧, ∨, ∃, ∀)
Order of operations matters	No order of operations

3.8 Family tree of query languages

```
              Relational Query Languages
                  |                  |
            Procedural          Non-procedural
          (HOW to compute)      (WHAT to retrieve)
                  |                  |
        Relational Algebra   Relational Calculus
                               |            |
                             TRC          DRC
                          (tuples)     (domains)
                               |            |
                             SQL          QBE
```

4. Real-Life Use

> **In real software projects**
>
> SQL is a practical descendant of TRC. `SELECT Name FROM Student WHERE CGPA > 8` is the TRC query `{ t.Name | t ∈ Student ∧ t.CGPA > 8 }`. You describe the result and the database engine picks the best plan. This is the idea behind declarative querying.

5. Exam Tips

> **Remember**
>
> - Learn the two **general forms** exactly: `{ t | P(t) }` and `{ <x1..xn> | P }`.
> - Give **at least two queries in each**, and include one with ∃.
> - The TRC vs DRC table and the "SQL is based on TRC" line are marks-winners.

Question 9: Normal Forms: 1NF, 2NF, 3NF, BCNF

> **The question (typos fixed)**
>
> Explain 1NF, 2NF, 3NF and BCNF with examples.
>
> **Unit:** Normalization (the most frequent 10-marker)

1. Understand It First (Plain English)

> **Cleaning a messy cupboard in four passes**
>
> Normal forms are **four cleaning passes** over a table. Each pass removes one kind of mess (redundancy). A table that passes all four is clean.
>
> - **1NF:** one value per box. No lists inside a cell.
> - **2NF:** every detail must depend on the **whole** key, not just a piece of it.
> - **3NF:** details must depend on the key and **nothing but the key** (no middle-man).
> - **BCNF:** every "decider" column must be a key.
>
> Words you need: a **candidate key** is the smallest set of columns that identifies a row. **Prime attribute** = a column that is part of some candidate key. **Non-prime** = a column that is not. A **superkey** is any set that identifies a row (a key plus possibly extra columns).

2. The 2-Mark Answer (write this)

> **2-Mark Answer**
>
> - **1NF:** all attributes hold atomic (single) values; no repeating groups.
> - **2NF:** in 1NF and no partial dependency (every non-prime attribute depends on the whole candidate key).
> - **3NF:** in 2NF and no transitive dependency. For every FD $X \to A$, either X is a superkey or A is a prime attribute.
> - **BCNF:** for every non-trivial FD $X \to Y$, X must be a superkey.

3. The 10-Mark Answer (full write-up)

3.1 First Normal Form (1NF)

Rule: every cell contains a **single atomic value**. No multi-valued attributes, no repeating groups.

Not in 1NF (Ravi has two phones in one cell):

RollNo	Name	Phones
1	Ravi	98765, 91234
2	Sita	99999

Converted to 1NF (one phone per row):

RollNo	Name	Phone
1	Ravi	98765
1	Ravi	91234
2	Sita	99999

(Even better design: a separate table `Student_Phone(RollNo, Phone)`, so Ravi's name is not repeated.)

3.2 Second Normal Form (2NF)

Rule: the table is in 1NF **and** every non-prime attribute is **fully dependent on the whole candidate key** (no partial dependency). This matters only when the key has **more than one column**.

Not in 2NF: ENROLLMENT(**RollNo, CourseID**, StudentName, Grade). The key is `{RollNo, CourseID}`.

RollNo	CourseID	StudentName	Grade
1	C1	Ravi	A
1	C2	Ravi	B
2	C1	Sita	B

FDs: `{RollNo, CourseID} → Grade` (full, fine) and `RollNo → StudentName` (**partial**: depends on only part of the key). Ravi's name is repeated, which causes update anomalies.

Fix: split the table so each part depends on its whole key.

RollNo (PK)	StudentName
1	Ravi
2	Sita

RollNo	CourseID	Grade
1	C1	A
1	C2	B
2	C1	B

3.3 Third Normal Form (3NF)

Rule: the table is in 2NF **and** has **no transitive dependency** of a non-prime attribute on the key. Formal rule: for every non-trivial FD `X → A`, **either X is a superkey, or A is a prime attribute**.

Not in 3NF: STUDENT(**RollNo**, Name, DeptID, DeptName).

RollNo	Name	DeptID	DeptName
1	Ravi	D1	CSE

RollNo	Name	DeptID	DeptName
2	Sita	D1	CSE
3	Amit	D2	ECE

FDs: `RollNo → DeptID`, `DeptID → DeptName`. So `RollNo → DeptName` is **transitive** (through DeptID). "CSE" is repeated.

Fix:

RollNo (PK)	Name	DeptID (FK)
1	Ravi	D1
2	Sita	D1
3	Amit	D2

DeptID (PK)	DeptName
D1	CSE
D2	ECE

3.4 Boyce-Codd Normal Form (BCNF)

Rule: for every non-trivial FD `X → Y`, **X must be a superkey**. BCNF is stricter than 3NF because it removes the "or A is prime" escape clause.

Example: TEACH(**Student, Subject**, Teacher). Rules: each teacher teaches only **one** subject; a student learns a subject from only one teacher.

Student	Subject	Teacher
S1	DBMS	Prof. X
S2	DBMS	Prof. X
S1	OS	Prof. Y

- FDs: `{Student, Subject} → Teacher` and `Teacher → Subject`.
- Candidate keys: `{Student, Subject}` and `{Student, Teacher}`.
- Check `Teacher → Subject`: Teacher is **not** a superkey, so the table is **not in BCNF**.
- But Subject is a prime attribute (part of a key), so the table **is in 3NF**.
- Problem: "Prof. X teaches DBMS" is stored twice. If Prof. X changes subject, several rows must change.

Fix (BCNF decomposition):

Teacher (PK)	Subject
Prof. X	DBMS
Prof. Y	OS

Student	Teacher
S1	Prof. X
S2	Prof. X
S1	Prof. Y

The split is **lossless** (the common column Teacher is a key of the first table). The cost: the FD `{Student, Subject} → Teacher` can no longer be checked inside one table. BCNF always gives a lossless split but **may not preserve dependencies**; 3NF can always do both.

3.5 How to check which normal form a table is in (step-by-step recipe)

1. Find all **candidate keys** (using attribute closure).
2. Mark **prime** attributes (in some key) and **non-prime** attributes (in no key).
3. **1NF:** are all values atomic?
4. **2NF:** is any non-prime attribute dependent on only **part** of a candidate key? If yes, not 2NF.
5. **3NF:** for every FD `X → A`, is X a superkey **or** A prime? If some FD fails both, not 3NF.
6. **BCNF:** for every FD `X → Y`, is X a superkey? If any fails, not BCNF.

3.6 Comparison table

Normal form	Removes	Condition
1NF	Multi-valued / composite attributes	Only atomic values
2NF	Partial dependency	Non-prime attributes depend on the **whole** key
3NF	Transitive dependency	For every FD X→A: X is a superkey **or** A is prime
BCNF	Anomalies left by non-superkey determinants	For every FD X→Y: X is a superkey

3.7 Progress diagram

```
Unnormalized table
      |  remove repeating groups / multi-valued cells
      v
     1NF
      |  remove partial dependencies
      v
     2NF
      |  remove transitive dependencies
      v
     3NF
      |  every determinant must be a superkey
      v
    BCNF        (4NF removes MVDs, 5NF removes join dependencies)

Containment:   BCNF  is inside  3NF  is inside  2NF  is inside  1NF
```

4. Real-Life Use

> **In real software projects**
>
> In a ride-sharing app, the `rides` table stores only a `driver_id`. The driver's name and car model live in a `drivers` table (3NF). If a driver changes car, **one row** changes. Most production transaction databases are designed to **3NF**, or BCNF where practical.

5. Exam Tips

> **Remember**
>
> - For every form: **rule, bad table, why it is bad, fixed tables**. Draw the tables.
> - 3NF vs BCNF is the favourite follow-up: write "**BCNF removes the 'A is prime' exception**".
> - Mention the progress diagram and the inclusion BCNF $\subset$ 3NF $\subset$ 2NF $\subset$ 1NF.

Question 10: Minimal Cover

The question (typos fixed)

Define minimal cover. Suppose a relation R(A, B, C) has the FD set F = { A → B, B → C, A → C, AB → B, AB → C, AC → B }. Find the minimal cover.

Unit: Functional Dependency Theory

1. Understand It First (Plain English)

Trimming a list of rules to the bare minimum

A set of FDs can contain **repeated or unnecessary** rules. A **minimal cover** is the shortest, cleanest version of the list that still means exactly the same thing. Like shortening "A→B, B→C, and also A→C" to just "A→B, B→C", because A→C already follows.

The tool you need is **attribute closure**, written X^+: the set of **all columns you can reach starting from X** using the FDs.

How to compute X^+: start with X. Look at each FD; if its left side is already inside your set, add its right side. Repeat until nothing new appears.

Example: with F = {A→B, B→C}, compute A^+. Start {A}. A→B adds B: {A, B}. B→C adds C: {A, B, C}. So A^+ = {A, B, C}.

2. The 2-Mark Answer (write this)

2-Mark Answer

A **minimal cover (canonical cover)** Fc of an FD set F is an equivalent set of FDs in which: (1) every right-hand side has a **single attribute**, (2) no left-hand side has an **extraneous attribute**, and (3) **no FD is redundant**.

For the given problem: Fc = **{ A → B, B → C }**.

3. The 10-Mark Answer (full write-up)

3.1 Definition

A **minimal cover** (also called canonical cover) of F is a set Fc such that (a) Fc is **equivalent** to F (it implies exactly the same dependencies), and (b) Fc is as small as possible:

1. Every FD has **only one attribute on the right side**.
2. No FD has an **extraneous (unnecessary) attribute** on the left side.
3. No FD is **redundant** (i.e. none can be derived from the others).

Terms: An attribute on the left side is extraneous if removing it does not change what the FD set can derive. An FD is redundant if it can be derived from the other FDs.

3.2 Algorithm

1. **Split:** make the right side of every FD a single attribute (use decomposition: `X → YZ` becomes `X → Y` and `X → Z`).
2. **Remove extraneous attributes** from each left side. To test whether attribute B is extraneous in `AB → C`: compute the closure of `A` (the left side without B) using F. If it contains C, then B is extraneous.
3. **Remove redundant FDs.** For each FD, temporarily delete it and compute the closure of its left side using the remaining FDs. If the right side is still reached, the FD is redundant; delete it for good.

3.3 Solution of the given problem

Given: R(A, B, C), F = { A → B, B → C, A → C, AB → B, AB → C, AC → B }

Step 1: single attribute on the right side. All FDs already have one attribute on the right. No change.

Step 2: remove extraneous attributes on the left. Only the FDs with two attributes on the left need checking: `AB → B`, `AB → C`, `AC → B`.

First, find A^+ using F: start {A}; A→B gives {A, B}; B→C gives {A, B, C}. So **A^+ = {A, B, C}**.

- **AB → B:** B is on both sides, so this is a **trivial FD** (right side is part of left side). It tells us nothing. **Delete it.**
- **AB → C:** is B extraneous? Remove B and test A: A^+ = {A, B, C} contains C. So **B is extraneous** and `AB → C` becomes **A → C**.
- **AC → B:** is C extraneous? Remove C and test A: A^+ contains B. So **C is extraneous** and `AC → B` becomes **A → B**.

Now the set is { A → B, B → C, A → C } (the duplicates collapse into one).

Step 3: remove redundant FDs. Test each FD in turn:

FD tested	Remaining FDs	Closure of its left side	Reached?	Decision
A → C	{A→B, B→C}	A^+ = {A, B, C}, contains C	Yes	**Redundant: remove**
A → B	{B→C}	A^+ = {A}	No	Keep
B → C	{A→B}	B^+ = {B}	No	Keep

3.4 Final answer

> **Minimal cover**
>
> **Fc = { A → B, B → C }**

Check (equivalence): from Fc we get `A → C` by transitivity. `AB → B` is trivial. `AB → C` and `AC → B` follow from A→C and A→B by augmentation. So Fc implies every FD in F, and Fc is a subset of F^+. Hence Fc ≡ F.

Bonus: A^+ = {A, B, C} = R, so **A is the candidate key** of R.

3.5 Flow of the solution

```
F = { A->B, B->C, A->C, AB->B, AB->C, AC->B }
          |
Step 1    |  single attribute on RHS        -> no change
          v
```

```
Step 2     |  AB->B : trivial                -> delete
           |  AB->C : B extraneous           -> A->C
           |  AC->B : C extraneous           -> A->B
           v
       { A->B, B->C, A->C }
           |
Step 3     |  A->C derivable from A->B, B->C -> delete
           v
      Fc = { A->B, B->C }        chain:  A --> B --> C
```

4. Real-Life Use

> **In real software projects**
>
> Schema-design tools use the minimal cover to create a **3NF design**: each FD in the cover becomes a table. Here you get R1(**A**, B) and R2(**B**, C), the design a developer would naturally make (for example `orders → customer_id` and `customers → city`).

5. Exam Tips

> **Remember**
>
> - Write the **3 conditions** in the definition and the **3 steps** of the algorithm. Then solve step by step.
> - Always show the **closure computation** when you claim an attribute is extraneous or an FD is redundant. The closure is the proof.
> - Work in this order: split, then extraneous attributes, then redundant FDs. Changing the order can give a wrong answer.

Question 11: Serializability of a Schedule

> **The question (typos fixed)**
>
> Given a schedule S for transactions T1 and T2: S: R1(X) R2(Y) W1(Y) R1(Y) W1(X). Identify whether the given schedule is equivalent to a serial schedule or not.
>
> **Unit:** Transaction Concurrency Control and Serializability

1. Understand It First (Plain English)

> **Two cooks sharing one kitchen**
>
> Two transactions (cooks) may take turns using the database (kitchen). Running one fully and then the other is safe, and is called a **serial schedule**. Letting them **interleave** (switch back and forth) is faster, but may cause chaos.
>
> A **schedule** is just the order in which the operations of different transactions are executed. We ask: "Is this interleaved schedule **equivalent** to some safe one-after-another order?" If yes, it is called **serializable**, so it is safe.
>
> Notation: `R1(X)` = transaction T1 **reads** item X. `W1(Y)` = T1 **writes** item Y.

2. The 2-Mark Answer (write this)

> **2-Mark Answer**
>
> The schedule is **conflict serializable**. Its precedence graph has a single edge **T2 → T1** and **no cycle**. It is equivalent to the serial schedule **T2 followed by T1**.

3. The 10-Mark Answer (full write-up)

3.1 Key definitions

- **Serial schedule:** transactions run one after another, no interleaving.
- **Conflicting operations:** two operations that (1) belong to **different** transactions, (2) access the **same** data item, and (3) **at least one is a write**. Types: read-write, write-read, write-write. Two reads never conflict.
- **Conflict equivalent:** two schedules where every pair of conflicting operations is in the **same order**.
- **Conflict serializable:** conflict equivalent to some serial schedule.
- **Precedence (serialization) graph:** one node per transaction. Draw an edge `Ti → Tj` if an operation of Ti conflicts with a **later** operation of Tj.
- **Theorem:** a schedule is conflict serializable **if and only if its precedence graph has no cycle**.

3.2 Step 1: separate the transactions

- T1: R1(X), W1(Y), R1(Y), W1(X)

- T2: R2(Y)

3.3 Step 2: write the schedule in time order

Time	T1	T2
1	R1(X)	
2		R2(Y)
3	W1(Y)	
4	R1(Y)	
5	W1(X)	

3.4 Step 3: find the conflicting pairs

- **Item X:** only T1 touches X (R1(X) and W1(X)). Same transaction, so **no conflict**.
- **Item Y:** the operations are R2(Y), W1(Y), R1(Y).
  - R2(Y) then W1(Y): different transactions, one is a write, so it is a **conflict (read-write)**. Edge **T2 → T1**.
  - R2(Y) and R1(Y): both are reads, so **no conflict**.
  - W1(Y) and R1(Y): same transaction, so **no conflict**.

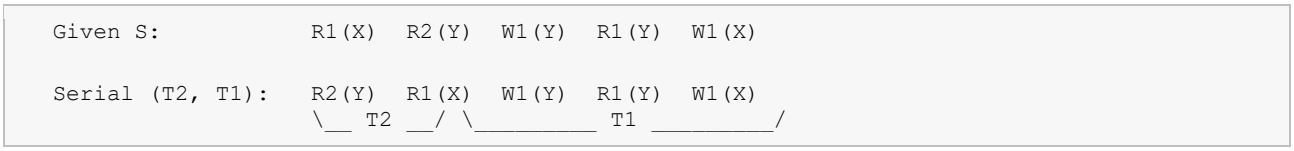
3.5 Step 4: draw the precedence graph



3.6 Step 5: conclusion

The graph has **no cycle**, so S is **conflict serializable**. The equivalent serial order is found by a topological sort of the graph: **T2 first, then T1**.

3.7 Step 6: verify by writing the serial schedule



The only conflicting pair is R2(Y) and W1(Y). In S, R2(Y) comes **before** W1(Y). In the serial schedule T2, T1, R2(Y) also comes **before** W1(Y). The order is the same, so the two schedules are **conflict equivalent**.

> **Final answer**
>
> S is **equivalent to the serial schedule T2 → T1**, hence it is **serializable** (and since conflict serializable implies view serializable, it is also view serializable).

3.8 Contrast: a schedule that is NOT serializable

To see why the cycle test matters, take S2: `R1(X) W2(X) W1(X)`.

- R1(X) before W2(X): read-write conflict, edge **T1 → T2**.
- W2(X) before W1(X): write-write conflict, edge **T2 → T1**.
- The graph has edges both ways, a **cycle**. So S2 is **not** serializable. Neither T1-then-T2 nor T2-then-T1 gives the same result.

```
   +----+  ---------->  +----+
   | T1 |               | T2 |      cycle  =>  NOT serializable
   +----+  <----------  +----+
```

3.9 Shortcut summary (how to do any such question)

1. List the operations of each transaction.
2. For each data item, find the conflicting pairs (different transactions, at least one write).
3. Draw an edge from the earlier transaction to the later one for each conflict.
4. No cycle: serializable, and the serial order is the topological order. Cycle: not serializable.

4. Real-Life Use

> **In real software projects**
>
> Databases set to the **SERIALIZABLE** isolation level guarantee results equal to some serial order. PostgreSQL does this with Serializable Snapshot Isolation, which watches these dependency edges and aborts one transaction if it finds a cycle (error "could not serialize access"). Applications must be ready to **retry** the transaction.

5. Exam Tips

> **Remember**
>
> - The expected structure: **separate transactions, list conflicts, draw the graph, state "no cycle" and give the serial order**.
> - Always draw the precedence graph. It is worth 3 or more marks by itself.
> - Do not mark two reads as a conflict, and do not mark operations of the same transaction as a conflict.

Question 12: Lossless Decomposition

> **The question (typos fixed)**
>
> Explain lossless decomposition with an example. A relation R(A, B, C, D, E) with FD set F = { A → BC, C → DE, D → E } is decomposed into R1(A, B, C, D) and R2(D, E). Check whether the decomposition is lossless or not.
>
> **Unit:** Decomposition: Lossless Join and Dependency Preservation

1. Understand It First (Plain English)

> **Cutting a photograph into two pieces**
>
> Normalization **splits** a big table into smaller ones (decomposition). After splitting, you must be able to **glue the pieces back (join)** and get **exactly the original table**. No missing rows, and **no fake extra rows**.
>
> If you can, the split is **lossless**. If gluing gives extra fake rows (called **spurious tuples**) the information is mixed up: that split is **lossy** and must be rejected.

2. The 2-Mark Answer (write this)

> **2-Mark Answer**
>
> A decomposition of R into R1 and R2 is **lossless** if `R1 ⋈ R2 = R` exactly, with no extra tuples and no lost information.
>
> **Test:** it is lossless if `(R1 ∩ R2) → R1` or `(R1 ∩ R2) → R2` holds (the common attributes form a key of one of the parts).
>
> **For the given problem:** R1 ∩ R2 = {D} and D → E, so D is a key of R2. The decomposition is **lossless**.

3. The 10-Mark Answer (full write-up)

3.1 Definition

Decomposition means replacing a relation R by two or more smaller relations R1, R2, ..., Rn. A decomposition is **lossless-join** if joining the parts always gives back the original relation: `R = R1 ⋈ R2 ⋈ ... ⋈ Rn`. It is **lossy** if the join gives extra (spurious) tuples.

3.2 Example showing a good and a bad split

Original table R(Emp, Dept, Mgr) with FD `Dept → Mgr` (each department has one manager). Here manager M1 manages two departments.

Emp	Dept	Mgr
E1	D1	M1
E2	D1	M1

Emp	Dept	Mgr
E3	D2	M1

Good (lossless) split: R1(Emp, Dept) and R2(Dept, Mgr). The common column is **Dept**, and `Dept → Mgr`, so Dept is a key of R2. Joining on Dept gives back exactly the 3 original rows.

Bad (lossy) split: R1(Emp, Mgr) and R2(Dept, Mgr). The common column is **Mgr**, which is **not a key** of either part.

R1: Emp	Mgr
E1	M1
E2	M1
E3	M1

R2: Dept	Mgr
D1	M1
D2	M1

Joining on Mgr pairs **every** employee with **every** department: 3 × 2 = 6 rows, but the original had only 3. For example (E1, D2, M1) is a **fake row**: E1 does not work in D2. These extra rows are spurious tuples, so the split is **lossy**.

3.3 The test for a split into two relations

Let R be split into R1 and R2 with FD set F. The decomposition is **lossless** if and only if:

1. `R1 ∪ R2 = R` (no attribute is dropped), and
2. `R1 ∩ R2 ≠ ∅` (there is a common attribute to join on), and
3. the common attributes determine one of the parts: `(R1 ∩ R2) → (R1 − R2)` **or** `(R1 ∩ R2) → (R2 − R1)` is in F^+. Equivalent to saying the common attributes are a **superkey of R1 or of R2**.

3.4 Solution of the given problem

Given: R(A, B, C, D, E), F = { A → BC, C → DE, D → E }. R1 = (A, B, C, D), R2 = (D, E).

Step 1: are all attributes covered? R1 ∪ R2 = {A, B, C, D, E} = R. Yes.

Step 2: common attributes. R1 ∩ R2 = {A, B, C, D} ∩ {D, E} = **{D}**. Not empty. Yes.

Step 3: closure of the common attribute. Start {D}. D → E adds E. So **D^+ = {D, E}**.

Step 4: compare. D^+ = {D, E} = R2. So D determines everything in R2, i.e. D is a **key of R2**, and `(R1 ∩ R2) → R2` holds.

Step 5: conclusion. The condition is satisfied, so the decomposition is **LOSSLESS**.

3.5 Second method: the tableau (chase) test

Make a table with one row per part and one column per attribute. Write **a** (a distinguished symbol) if the attribute belongs to that part, otherwise write a **b** (a placeholder).

Initial tableau:

	A	B	C	D	E
R1 (ABCD)	a1	a2	a3	a4	b15
R2 (DE)	b21	b22	b23	a4	a5

Now apply each FD. If two rows agree on the left side, make their right sides equal (always replace a **b** by the matching **a**):

- `A → BC`: A values are a1 and b21 (different), nothing to do.
- `C → DE`: C values are a3 and b23 (different), nothing to do.
- `D → E`: both rows have D = a4, so their E values must be equal: b15 and a5. **Set b15 = a5.**

Final tableau:

	A	B	C	D	E
R1 (ABCD)	a1	a2	a3	a4	**a5**
R2 (DE)	b21	b22	b23	a4	a5

The first row is now **all a-symbols** (a1 a2 a3 a4 a5). A row with all a-symbols proves the decomposition is **lossless**.

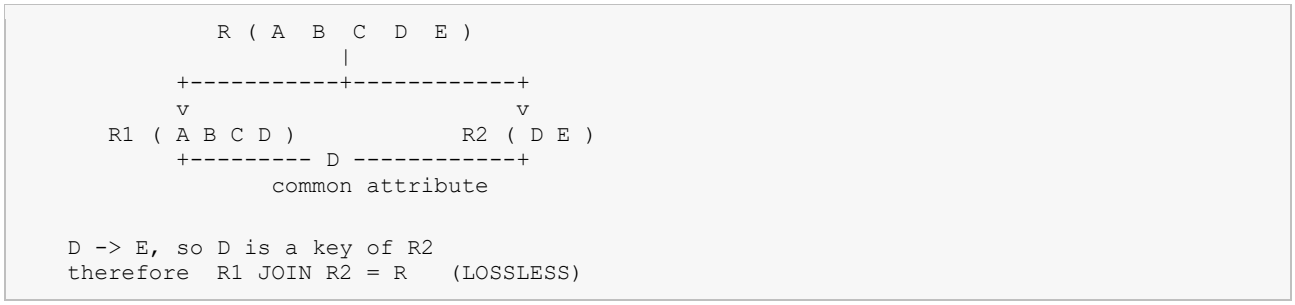
3.6 Bonus: dependency preservation

A good decomposition should also keep the original FDs checkable inside the parts (**dependency preserving**). Check each FD of F:

FD	Where it holds	Preserved?
A → BC	A, B, C are all in R1	Yes
C → DE	C → D holds in R1 and D → E holds in R2, so C → DE follows by combining them	Yes
D → E	D, E are both in R2	Yes

So the decomposition is **both lossless and dependency preserving**.

3.7 Diagram



4. Real-Life Use

In real software projects

When a developer splits a wide `orders` table into `orders` and `order_details`, the **join column** (such as `order_id`) must be a key in at least one of them. Otherwise the join duplicates or invents rows. This is the classic cause of the bug "my report totals are too big after a JOIN".

5. Exam Tips

Remember

- Write the **test** (R1 ∩ R2 is a key of R1 or R2) first, then apply it with the **closure** of the common attribute.
- Give a **good and a bad example** to explain the concept, and mention **spurious tuples**.
- Show the tableau method if you have time. It is a second proof and impresses examiners.

Quick Revision Sheet

Read this the night before and again just before the exam.

One-line answers

Q	Remember this
1	DDL = structure (CREATE, ALTER, DROP, TRUNCATE; auto-commit). DML = data (INSERT, UPDATE, DELETE, SELECT; can rollback).
2	Trigger = code that runs automatically on INSERT/UPDATE/DELETE. ECA model. BEFORE/AFTER, row/statement level. OLD and NEW.
3	Normalization removes redundancy and the 3 anomalies (insert, delete, update). Improves integrity, saves space.
4	FD types: trivial, non-trivial, completely non-trivial, full, partial, transitive, multivalued, join.
5	Constraints: NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK, DEFAULT. PK = UNIQUE + NOT NULL.
6	Basic: σ π ∪ − × ρ. Derived: ∩ ⋈ ÷. Union needs compatible relations. Projection removes duplicates.
7	ACID: Atomicity (all or nothing), Consistency (valid to valid), Isolation (no interference), Durability (permanent).
8	TRC: { t | P(t) }, variables are tuples, basis of SQL. DRC: { <x1..xn> | P }, variables are values, basis of QBE.
9	1NF atomic, 2NF no partial dependency, 3NF no transitive dependency, BCNF every determinant is a superkey.
10	Minimal cover = **{ A → B, B → C }**. Steps: split RHS, remove extraneous attributes, remove redundant FDs.
11	Precedence graph has only T2 → T1, no cycle, so serializable. Serial order: **T2, then T1**.
12	R1 ∩ R2 = {D}, D^+ = {D,E} = R2, so D is a key of R2. **Lossless**. Also dependency preserving.

Formulas and rules to memorise

Topic	Rule
Closure X^+	Start with X. Add RHS of every FD whose LHS is inside the set. Repeat until nothing changes.
Candidate key	Smallest set whose closure is all attributes.
3NF condition	For every FD X → A: X is a superkey OR A is a prime attribute.
BCNF condition	For every FD X → Y: X is a superkey.
Lossless (binary)	(R1 ∩ R2) → R1 or (R1 ∩ R2) → R2 must hold.
Conflict	Different transactions + same item + at least one write.
Serializable	Precedence graph has NO cycle.
Cartesian product size	m rows × n rows = m × n rows.

Last-minute checklist

- Did I write the **definition first**?
- Did I draw at least one **table or diagram**?
- Did I give an **example**?
- For numericals: did I show **every step** (closure, graph, tableau)?
- Did I use **headings** and underline key terms?